

LAB 9

NAME: AADITYAAMLAN PANDA

ROLL: 220007

INTRODUCTION

Using PLS the problem mentioned in the paper of Prof H. Abdi was solved to verify the results of the paper. The X and Y data are as follows: -

$$Y^{5 \times 3} = \begin{bmatrix} [14,7,8], \\ [10,7,6], \\ [8,5,5], \\ [2,4,7], \\ [6,2,4] \end{bmatrix}$$

$$X^{5 \times 3} = \begin{bmatrix} [7,7,13,7], \\ [4,3,14,7], \\ [10,5,12,5], \\ [16,7,11,3], \\ [13,3,10,3] \end{bmatrix}$$

METHODOLOGY

- Each column of X was converted to its Z score ($x_i - \text{mean} / \sqrt{\text{variance}}$) and called E. Same thing was done with Y and called F. Their sum of squares are calculated as SSX and SSY
- A guess vector $u^{5 \times 1}$ was taken with all elements 1.
- For a total of “iterlim” iterations the following were run
 - A nested second was used for the each “itermax” iteration till $t_{\text{old}} - t < 10^{-6}$ is satisfied
 - $w = E'u / \|E'u\|$ to calculate X weights
 - $t = Ew / \|Ew\|$ for X factor scores
 - $c = F't / \|F't\|$ for estimating Y weights
 - $u = Fc$ for Y scores
 - Individual factor loadings for Y, $b = t'u$ were calculated.
 - Individual factor loadings for X, $p = E't$ were calculated.
 - Var for each latent vectors were calculated using: -
 - Var Explained in X = $p'p / \text{SSX} * 100$
 - Var Explained in Y = $b^2 / \text{SSY} * 100$
 - Also Cumulative variances were calculated
 - u was appended to U , t to T , w to W , c to C , p to P and b was stored in diagonals of B
 - $F = F - btc'$ and $E = E - tp'$ for carrying out rank 1 decompositions on X and Y.
 - B_{PLS} was calculated using $B_{\text{PLS}} = (P'^+)BC'$, where P'^+ is the Moore-Penrose pseudo-inverse of P' , after both second and third latent vectors to verify the results
- After “iterlim” iterations, B_{PLS} was recalculated. Also $Y_{\text{hat}} = TBC'$ and $X_{\text{hat}} = TP'$

RESULTS (Upto 5 iterations)

Z-score of X:
[[-0.707 1.118 0.707 1.118]
[-1.414 -1.118 1.414 1.118]
[0. 0. 0. 0.]
[1.414 1.118 -0.707 -1.118]
[0.707 -1.118 -1.414 -1.118]]

Z-score of Y:
[[1.5 1.054 1.414]
[0.5 1.054 0.]
[0. 0. -0.707]
[-1.5 -0.527 0.707]
[-0.5 -1.581 -1.414]]

B_pls with 2 latent variables:

```
[[ -0.266 -0.25  0.012]
 [ 0.062  0.32  0.79 ]
 [ 0.297  0.368  0.257]
 [ 0.301  0.37  0.251]]
```

B_pls with 3 latent variables:

```
[[ -1.061 -0.075  0.125]
 [ 0.335  0.259  0.751]
 [ -1.414  0.745  0.5 ]
 [ 1.23  0.165  0.119]]
```

Final U:

```
[[ 2.175  1.045 -0.26 -1.024 -1.936]
 [ 0.851 -0.593 -0.68  1.294 -0.872]
 [ 0.692 -0.602  0.092 -0.686  0.505]
 [ 0.177  0.177 -0.707  0.177  0.177]
 [ 0.078  0.063 -0.707  0.202  0.221]]
```

Final T:

```
[[ 0.454  0.54  0.  -0.43 -0.563]
 [ 0.466 -0.494  0.   0.533 -0.505]
 [ 0.572 -0.463  0.  -0.53  0.422]
 [ 0.621  0.715  0.  -0.157 -0.281]
 [ 0.199  0.24  0.   0.665  0.679]]
```

Final W:

```
[[ -0.514  0.201  0.57  0.608]
 [ 0.338  0.94  0.019 -0.043]
 [ -0.349  0.161 -0.821  0.422]
 [ 0.524  0.357 -0.429 -0.643]
 [ 0.559  0.539  0.305  0.551]]
```

Final C:

```
[[ 0.609  0.702  0.368]
 [ -0.052  0.268  0.962]
 [ 0.967 -0.218 -0.13 ]
 [ -0.  -0.   1.  ]
 [ 0.   0.   1.  ]]
```

Final P:

```
[[ -2.091  0.052  2.185  2.222]
 [ 0.765  2.234 -0.032 -0.062]
 [ -0.201  0.093 -0.472  0.243]
 [ 0.   -0.   -0.   -0.  ]
 [ 0.   0.   0.   0.  ]]
```

Final B:

```
[[ 3.082 0.  0.  0.  0. ]
 [ 0.  1.819 0.  0.  0. ]
 [ 0.  0.  1.251 0.  0. ]
 [ 0.  0.  0.  0.159 0. ]
 [ 0.  0.  0.  0.  0.315]]
```

B_pls with 5 latent variables:

```
[[ -1.061 -0.075  0.125]
 [ 0.335  0.259  0.751]
 [ -1.414  0.745  0.5 ]
 [ 1.23  0.165  0.119]]
```

X fitted:

```
[[ -0.707  1.118  0.707  1.118]
 [ -1.414 -1.118  1.414  1.118]
 [ 0.   0.   0.   0. ]
 [ 1.414  1.118 -0.707 -1.118]
 [ 0.707 -1.118 -1.414 -1.118]]
```

Y fitted:

```
[[ 1.5  1.054  1.399]
 [ 0.5  1.054  0.012]
 [ 0.   0.   0. ]
 [ -1.5 -0.527  0.715]
 [ -0.5 -1.581 -1.422]]
```

Variance explained in X:

```
[70.451 27.896 1.654 0.  0. ]
```

Cumulative variance explained in X:

```
[ 70.451 98.346 100.  100.  100. ]
```

Variance explained in Y:

```
[63.332 22.064 10.437 0.168 0.662]
```

Cumulative variance explained in Y:

```
[63.332 85.396 95.833 96.001 96.663]
```

OBSERVATION

1. The B_{PLS} was obtained to be as given in the research paper.
2. With higher iterations it was seen that Variance in X was completely explained by latent vectors while that that in Y stabilises in around 96.67 %
3. X fitted and Y fitted exactly correponded to the original data points in Zscoresof X and Y, indicating that Y belongs to the column space of X.

APPENDIX

```

import numpy as np
np.set_printoptions(precision=3,
suppress=True)
def find_mean(X):
n = len(X)
s = 0.0
for i in range(n):
s += X[i]
return s/n

def find_variance(X, mean):
n = len(X)
s = 0.0
for i in range(n):
s += (X[i] - mean)**2
return s/n

def find_Z_score(X):
n,m = X.shape
Z = np.zeros((n,m))
for j in range(m):
mean = find_mean(X[:,j])
variance = find_variance(X[:,j], mean)
for i in range(n):
if variance!=0:
Z[i,j] = (X[i,j] - mean)/np.sqrt(variance)
else:
Z[i,j] = 0.0
return Z

def find_transpose(X):
n,m = X.shape
T = np.zeros((m,n))
for i in range(n):
for j in range(m):
T[j,i] = X[i,j]
return T

def find_w(E,u):
n,m = E.shape
w = np.zeros(m)
Et = find_transpose(E)
for i in range(m):
s = 0.0
for j in range(n):
s += Et[i,j]*u[j]
w[i] = s
return w

def find_t(E,w):

```

```

n,m = E.shape
t = np.zeros(n)
for i in range(n):
s = 0.0
for j in range(m):
s += E[i,j]*w[j]
t[i] = s
return t

```

```

def find_c(F,t):
n,m = F.shape
c = np.zeros(m)
Ft = find_transpose(F)
for i in range(m):
s = 0.0
for j in range(n):
s += Ft[i,j]*t[j]
c[i] = s
return c

```

```

def find_u(F,c):
n,m = F.shape
u = np.zeros(n)
for i in range(n):
s = 0.0
for j in range(m):
s += F[i,j]*c[j]
u[i] = s
return u

```

```

def find_p(E,t):
n,m = E.shape
p = np.zeros(m)
Et = find_transpose(E)
for i in range(m):
s = 0.0
for j in range(n):
s += Et[i,j]*t[j]
p[i] = s
return p

```

```

def find_b(t,u):
n = len(t)
s = 0.0
for i in range(n):
s += t[i]*u[i]
return s

```

```

def find_diff_norm(t,t_old):

```

```

n = len(t)
s = 0.0
for i in range(n):
    s += (t[i] - t_old[i])**2
return np.sqrt(s)

```

```

def find_norm(t):
    n = len(t)
    s = 0.0
    for i in range(n):
        s += t[i]**2
    return np.sqrt(s)

```

```

def find_outer_product(x,y):
    n = len(x)
    m = len(y)
    op = np.zeros((n,m))
    for i in range(n):
        for j in range(m):
            op[i,j] = x[i]*y[j]
    return op

```

```

def dot_product(x,y):
    n = len(x)
    s = 0.0
    for i in range(n):
        s += x[i]*y[i]
    return s

```

```

def matrix_multiply(A,B):
    n = A.shape[0]
    m = B.shape[1]
    p = A.shape[1]
    C = np.zeros((n,m))
    for i in range(n):
        for j in range(m):
            s = 0.0
            for k in range(p):
                s += A[i,k]*B[k,j]
            C[i,j] = s
    return C

```

```

def sum_of_squares(X):
    n,m = X.shape
    s = 0.0
    for i in range(n):
        for j in range(m):
            s += X[i,j]**2
    return s

```

```

Y = np.array([[14,7,8],
              [10,7,6],
              [8,5,5],
              [2,4,7],
              [6,2,4]])

```

```

X = np.array([[7,7,13,7],
              [4,3,14,7],
              [10,5,12,5],
              [16,7,11,3],
              [13,3,10,3]])

```

```

E = X.copy()
F = Y.copy()
E = find_Z_score(E)
F = find_Z_score(F)
print("Z-score of X:\n", E)
print("Z-score of Y:\n", F)

```

```

SSX = sum_of_squares(E)
SSY = sum_of_squares(F)

```

```

iterlim = 5
U = []
T = []
W = []
C = []
P = []
B = np.zeros((iterlim,iterlim))
Var = np.zeros((iterlim,4))

```

```

for itermx in range(iterlim):
    u = np.ones(E.shape[0])
    iter = 0
    t = u.copy()
    while True:
        t_old = t.copy()
        u_old = u.copy()
        w = find_w(E,u)
        w = w/find_norm(w)
        t = find_t(E,w)
        t = t/find_norm(t)
        c = find_c(F,t)
        c = c/find_norm(c)
        u = find_u(F,c)
        if find_diff_norm(t,t_old) < 1e-6 and
           find_diff_norm(u,u_old) < 1e-6:
            break
        iter += 1
    b = find_b(t,u)
    p = find_p(E,t)

```

```

Var[itermax,0] = dot_product(p,p)/SSX*100
Var[itermax,2] = b**2/SSY*100
if itermax > 0:
    Var[itermax,1] = Var[itermax-1,1] +
    Var[itermax,0]
    Var[itermax,3] = Var[itermax-1,3] +
    Var[itermax,2]
else:
    Var[itermax,1] = Var[itermax,0]
    Var[itermax,3] = Var[itermax,2]
    U.append(u.copy())
    T.append(t.copy())
    W.append(w.copy())
    C.append(c.copy())
    P.append(p.copy())
    B[itermax,itermax] = b
    E = E - find_outer_product(t,p)
    F = F - b*find_outer_product(t,c)

if itermax == 1:
    B_pls_2 =
    matrix_multiply(np.linalg.pinv(find_transpose(
    np.column_stack(P))),matrix_multiply(B[0:2,0:2
    ],find_transpose(np.column_stack(C))))
    print("B_pls with 2 latent variables:\n",
    B_pls_2)

if itermax == 2:
    B_pls_3 =
    matrix_multiply(np.linalg.pinv(find_transpose(
    np.column_stack(P))),matrix_multiply(B[0:3,0:3
    ],find_transpose(np.column_stack(C))))

```

```

print("B_pls with 3 latent variables:\n",
B_pls_3)

```

```

print("Final U:\n", np.array(U))
print("Final T:\n", np.array(T))
print("Final W:\n", np.array(W))
print("Final C:\n", np.array(C))
print("Final P:\n", np.array(P))
print("Final B:\n", np.array(B))

```

```

B_pls =
matrix_multiply(np.linalg.pinv(find_transpose(
np.column_stack(P))),matrix_multiply(B,find_tr
anspose(np.column_stack(C))))
print("B_pls with " + str(iterlim) + " latent
variables:\n", B_pls)

```

```

X_hat =
matrix_multiply(np.column_stack(T),find_trans
pose(np.column_stack(P)))
Y_hat =
matrix_multiply(np.column_stack(T),matrix_m
ultiply(B,find_transpose(np.column_stack(C))))

print("X fitted:\n", X_hat)
print("Y fitted:\n", Y_hat)

```

```

print("Variance explained in X:\n", Var[:,0])
print("Cumulative variance explained in X:\n",
Var[:,1])
print("Variance explained in Y:\n", Var[:,2])
print("Cumulative variance explained in Y:\n",
Var[:,3])

```